Chapter 9: Database Security: An Introduction

Group 6

February 12, 2026



Main contents

1. Introduction to Database Security Issues

2. Discretionary
Access Control (DAC)

3. Mandatory
Access Control (MAC)

4. Role-Based Access Control (RBAC)

5. Attribute-Based
Access Control (ABAC)

6. Context-Based
Access Control (CBAC)

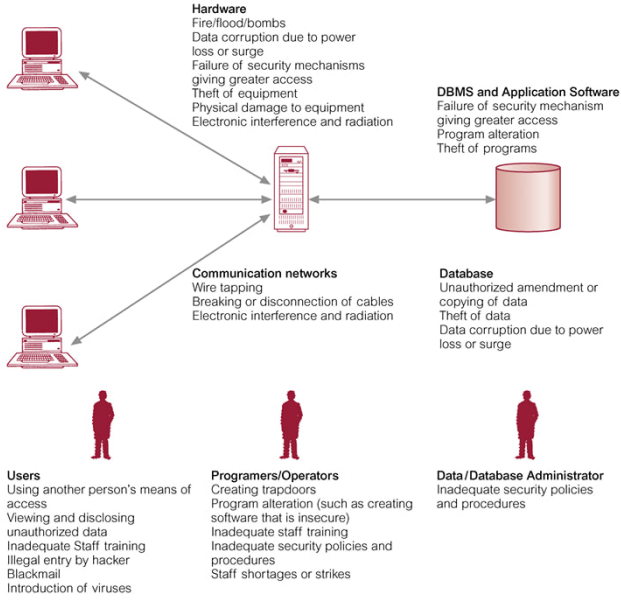
Introduction to Database Security Issues

- **Types of Security:**
  - **Legal and ethical issues:** *e.g., GDPR, HIPAA regulations on personal data protection.*
  - **Policy issues:** *e.g., company rules on who can access payroll data.*
  - **System-related issues:** *e.g., choosing security at hardware, OS, or DBMS level.*
  - **Multiple security levels:** *e.g., classifying data as Public, Confidential, or Top Secret.*

- **Three Basic Concepts:**
  - **Authentication** *a* mechanism that determines whether a user is who he or she claims to be.
  - **Authorization** *t*he granting of a right or privilege, which enables a subject to legitimately have access to a system or a system's objects.
  - **Access Control** *a* security mechanism (of a DBMS) for restricting access to a system's objects (the database) as a whole.

- **Threats:**
  - Any situation or event, whether intentional or unintentional, that will adversely affect a system and consequently an organization.
  - Threats to:
    - Computer systems
    - Databases

Threats to Computer Systems



- **Threats to databases:**
  - Loss of **integrity**
  - Loss of **availability**
  - Loss of **confidentiality**
- To protect databases against these types of threats four kinds of countermeasures can be implemented:
  - **Access control**
  - **Inference control**
  - **Flow control**
  - **Encryption**

Introduction to Database Security Issues (5)

- A DBMS typically includes a database security and authorization subsystem that is responsible for ensuring the security portions of a database against unauthorized access.

- Two types of database security mechanisms:
  - **Discretionary** security mechanisms
    - *Example:* The owner of a table grants `SELECT` privilege to specific users via `GRANT SELECT ON Employee TO Alice;`
  - **Mandatory** security mechanisms
    - *Example:* Data is classified as *Top Secret* / *Secret* / *Public*; a user with *Secret* clearance cannot read *Top Secret* data, regardless of the owner's wish.

- The security mechanism of a DBMS must include provisions for restricting access to the database as a whole
  - This function is called **access control** and is handled by creating user accounts and passwords to control login process by the DBMS.

- The security problem associated with databases is that of controlling the access to a **statistical database**, which is used to provide statistical information or summaries of values based on various criteria.
  - The countermeasures to **statistical database security** problem is called **inference control measures**.

- **Example:** A university database stores individual salary records but only allows queries like *AVG*, *SUM*, *COUNT* on groups of employees.
- **Security vulnerability - Inference Attack:**
  - Query 1: `SELECT COUNT(*) FROM Emp WHERE Dept='CS' AND Gender='F'`
    $\rightarrow$ Result: **1**
  - Query 2: `SELECT AVG(Salary) FROM Emp WHERE Dept='CS' AND Gender='F'`
    $\rightarrow$ Returns the salary of that **one specific person**!
  - $\Rightarrow$ Attacker infers **individual data** from aggregate queries.

- **Common techniques to prevent inference attacks:**
  - **Query Set Size Control:** reject queries whose result set is too small (e.g., less than 5 records).
  - **Query Restriction / Auditing:** log queries and block suspicious query sequences.
  - **Data Perturbation:** slightly modify data values (add noise) before returning results.
  - **Data Swapping:** exchange values between records while preserving statistical properties.
  - **Differential Privacy:** add mathematically calibrated noise to guarantee individual privacy (modern approach used by Apple, Google).

- Another security is that of **flow control**, which prevents information from flowing in such a way that it reaches unauthorized users.
  - *Example:* A low-clearance user reads a "Public" report, but the report was generated from "Top Secret" data $\rightarrow$ information has **leaked** through an improper flow.

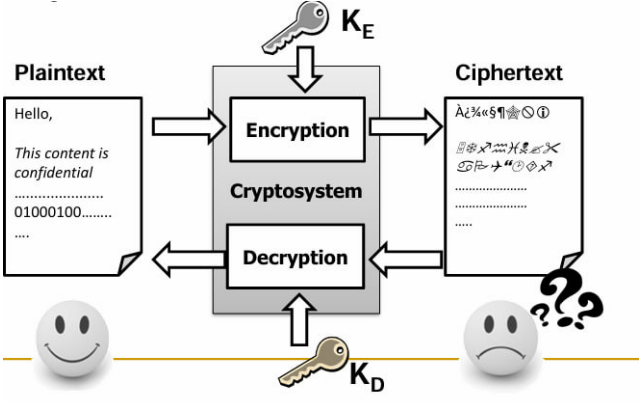
- Channels that are pathways for information to flow implicitly in ways that violate the security policy of an organization are called **covert channels**.
  - *Example:* A high-privilege process encodes secret data by **varying CPU usage** or **locking/unlocking a shared file** in a pattern. A low-privilege process observes the pattern and decodes the secret — **without ever directly reading it**.

Introduction to Database Security Issues (10)

- A final security issue is **data encryption**, which is used to protect sensitive data (such as credit card numbers) that is being transmitted via some type communication network.

- The data is **encoded** using some **encoding algorithm**.
  - An unauthorized user who access encoded data will have difficulty deciphering it, but authorized users are given decoding or decrypting algorithms (or keys) to decipher data.

- **Encryption** protects sensitive data by encoding it so that only authorized users with the correct key can decipher it.



- **Two main protection goals:**
  - **Data at-rest:** protecting data stored on disks, backups, or database files.
    - *Example:* Encrypting the entire database file using **Transparent Data Encryption (TDE)** in SQL Server / Oracle.
  - **Data in-transit:** protecting data while being transmitted over networks.
    - *Example:* Using **TLS/SSL** when a client connects to a database server.

Data Encryption (3) - Classic Techniques

- **Symmetric Encryption:** same key for encryption and decryption.
  - **AES** (Advanced Encryption Standard) - industry standard, 128/192/256-bit.
  - **DES / 3DES** - older, largely replaced by AES.
- **Asymmetric Encryption:** uses a public key + private key pair.
  - **RSA** - widely used for key exchange and digital signatures.
  - **ECC** (Elliptic Curve Cryptography) - stronger security with smaller keys.
- **Hashing:** one-way function for password storage and integrity check.
  - **SHA-256**, **bcrypt**, **Argon2**.

- **Homomorphic Encryption:**
  - Allows computation **directly on encrypted data** without decrypting it.
  - *Example:* A cloud server can compute `SUM(salary)` on encrypted data and return an encrypted result — the server never sees the plaintext.
- **End-to-End Encryption (E2EE):**
  - Data is encrypted on the sender's device and only decrypted on the receiver's device (e.g., Signal, WhatsApp).
- **Post-Quantum Cryptography:**
  - New algorithms (e.g., **CRYSTALS-Kyber**, **Dilithium**) designed to resist attacks from future **quantum computers**.
  - Standardized by NIST in 2024.

Database Security and the DBA (1)

- The database administrator (**DBA**) is the central authority for managing a database system.
  - The DBA's responsibilities include:
    - granting privileges to users who need to use the system.
    - classifying users and data in accordance with the policy of the organization.
- The DBA is responsible for the overall security of the database system.

Database Security and the DBA (2)

- The DBA has a DBA account in the DBMS:
  - Sometimes these are called a system or superuser account.
  - These accounts provide powerful capabilities such as:
    - 1. Account creation
    - 2. Privilege granting
    - 3. Privilege revocation
    - 4. Security level assignment
  - Action 1 is access control, whereas 2 and 3 are discretionary and 4 is used to control mandatory authorization.

Access Protection, User Accounts, and Database Audits (1)

- Whenever a person or group of persons need to access a database system, the individual or group must first apply for a user account.
  - The DBA will then create a new **account id** and **password** for the user if he/she deems there is a legitimate need to access the database.
- The user must log in to the DBMS by entering account id and password whenever database access is needed.

- The database system must also keep **track of all operations** on the database that are applied by a certain user throughout **each login session**.
  - To keep a record of all updates applied to the database and of the particular user who applied each update, we can modify **system log**, which includes an entry for each operation applied to the database that may be required for recovery from a transaction failure or system crash.

- If any tampering with the database is suspected, a **database audit** is performed.
  - A database audit consists of reviewing the log to examine all accesses and operations applied to the database during a certain time period.
- A database log that is used mainly for security purposes is sometimes called an **audit trail**.

Contents

1. Introduction to Database Security Issues
2. **Discretionary Access Control (DAC)**
3. Mandatory Access Control (MAC)
4. Role-Based Access Control (RBAC)
5. Attribute-Based Access Control (ABAC)
6. Context-Based Access Control (CBAC)

Discretionary Access Control (DAC)

Discretionary Access Control (DAC)

- User can protect what they own.
- Owner may grant access to other.
- Owner can define the type of access (read/write/execute/...) given to others.

- The typical method of enforcing **discretionary access control** in a database system is based on the **granting** and **revoking privileges**.

Types of Discretionary Privileges (1)

- The **account level**:
  - At this level, the DBA specifies the particular privileges that each account holds independently of the relations in the database.
- The **relation level** (or **table level**):
  - At this level, the DBA can control the privilege to access each individual relation or view in the database.

Types of Discretionary Privileges (2)

- The privileges at the **account level** apply to the capabilities provided to the account itself and can include:
  - the **CREATE SCHEMA** or **CREATE TABLE** privilege, to create a schema or base relation;
  - the **CREATE VIEW** privilege;
  - the **ALTER** privilege, to apply schema changes such adding or removing attributes from relations;
  - the **DROP** privilege, to delete relations or views;
  - the **MODIFY** privilege, to insert, delete, or update tuples;
  - and the **SELECT** privilege, to retrieve information from the database by using a **SELECT** query.

Types of Discretionary Privileges (3)

- The second level of privileges applies to the **relation level**
  - This includes **base relations** and virtual (**view**) relations.
- In SQL the following types of privileges can be granted on each individual relation R:
  - **SELECT** (retrieval or read) privilege on R:
    - This gives the account retrieval privilege.
    - The **SELECT** statement is used to retrieve tuples from R.
  - **REFERENCES** privilege on R:
    - This gives the account the capability to **reference** relation R when specifying integrity constraints.
    - The privilege can also be **restricted** to specific attributes of R.

Types of Discretionary Privileges (4)

- In SQL the following types of privileges can be granted on each individual relation R (contd.):
  - **MODIFY** privileges on R:
    - This gives the account the capability to modify tuples of R.
    - In SQL this privilege is further divided into **UPDATE**, **DELETE**, and **INSERT** privileges to apply the corresponding SQL command to R.
    - In addition, both the **INSERT** and **UPDATE** privileges can specify that only certain attributes can be updated by the account.

- Notice that to create a **view**, the account must have **SELECT** privilege on all relations involved in the view definition.

Types of Discretionary Privileges (5)

- The granting and revoking of privileges generally follow an authorization model for discretionary privileges known as the access matrix model where:
  - The **rows** of a matrix M represents **subjects** (users, accounts, programs)
  - The **columns** represent **objects** (relations, records, columns, views, operations).
  - Each position **M(i,j)** in the matrix represents the types of privileges (read, write, update) that **subject i** holds on **object j**.

Access

matrix

model

	O1	…	Oi	…	Om
S1	A[s1,o1]		A[s1,oi]		A[s1,om]
…					
Si	A[si,o1]		A[si,oi]		A[si,om]
…					
Sn	A[sn,o1]		A[sn,oi]		A[sn,om]

Types of Discretionary Privileges (6)

- To control the granting and revoking of relation privileges, for each relation R in a database:
  - The owner of a relation is given **all** privileges on that relation.
  - The owner account holder can **pass privileges** on any of the owned relation to other users by **granting** privileges to their accounts.
  - The owner account holder can also **take back** the **privileges** by **revoking** privileges from their accounts.

- The mechanism of **views** is an important discretionary authorization mechanism in its own right.
  - If the owner A of a relation R wants another account B to be able to **retrieve only some fields** of R, then A can **create a view** V of R that includes **only those attributes** and then grant SELECT on V to B.
  - The same applies to limiting B to retrieving **only certain tuples of** R; a view V' can be created by defining the view by means of a query that selects only those tuples from R that A wants to allow B to access.

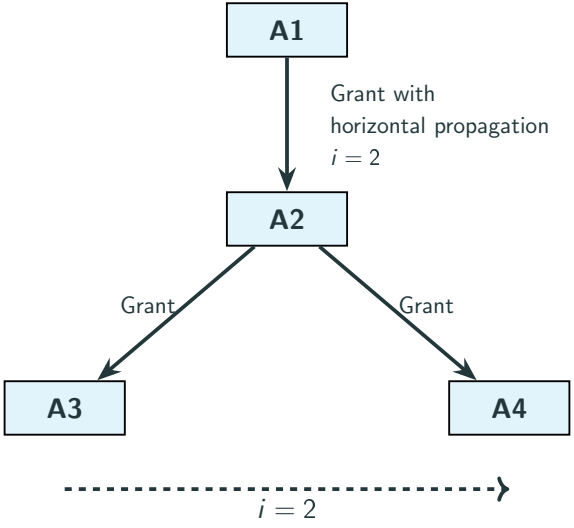
Revoking Privileges

- In some cases it is desirable to grant a privilege to a user temporarily.
  - The owner of a relation may want to grant the **SELECT** privilege to a user for a specific task and then revoke that privilege once the task is completed.
  - Hence, a mechanism for **revoking** privileges is needed. In SQL, a **REVOKE** command is included for the purpose of **canceling privileges**.

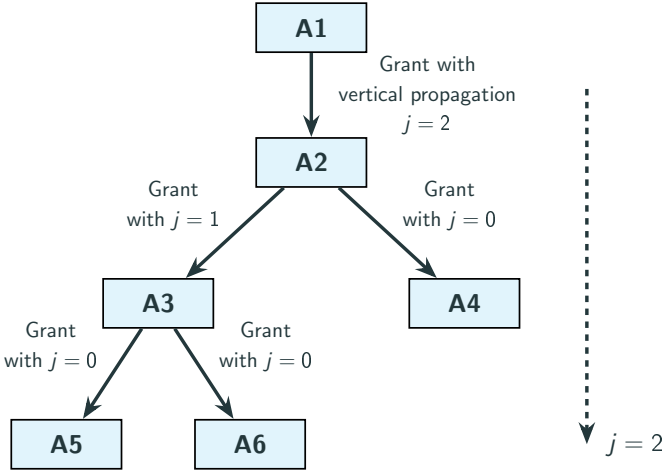
Propagation of Privileges using the GRANT OPTION

- Whenever the owner A of a relation R grants a privilege on R to another account B, privilege can be given to B with or without the **GRANT OPTION**.
- If the **GRANT OPTION** is given, this means that B can also grant that privilege on R to other accounts.
  - Suppose that B is given the **GRANT OPTION** by A and that B then grants the privilege on R to a third account C, also with **GRANT OPTION**. In this way, privileges on R can **propagate** to other accounts without the knowledge of the owner of R.
  - If the owner account **A now revokes** the privilege granted to B, **all the privileges that B propagated** based on that privilege should automatically **be revoked** by the system.

Limiting the horizontal propagation



Limiting the vertical propagation



An Example (1)

- Suppose that the DBA creates four accounts:
  - A1, A2, A3, A4
- and wants only A1 to be able to create base relations. Then the DBA must issue the following GRANT command in SQL:

```
GRANT CREATETAB TO A1;
```

- In SQL2 the same effect can be accomplished by having the DBA issue a **CREATE SCHEMA** command as follows:

```
CREATE SCHEMA EXAMPLE AUTHORIZATION A1;
```

An Example (2)

- User account **A1 can create tables** under the schema called **EXAMPLE**.
- Suppose that A1 **creates** the two base relations **EMPLOYEE** and **DEPARTMENT**:
  - A1 is then **owner** of these two relations and hence **all the relation privileges** on each of them.
- Suppose that A1 wants to grant A2 the privilege to insert and delete tuples in both of these relations, but A1 does not want A2 to be able to propagate these privileges to additional accounts:

```
GRANT INSERT, DELETE ON
        EMPLOYEE, DEPARTMENT TO A2;
```

EMPLOYEE

Name	Ssn	Bdate	Address	Sex	Salary	Dno

DEPARTMENT

Dnumber	Dname	Mgr_ssn

Figure 24.1
Schemas for the two relations EMPLOYEE and DEPARTMENT.

An Example (4)

- Suppose that A1 wants to allow A3 to retrieve information from either of the two tables and also to be able to propagate the SELECT privilege to other accounts.
- A1 can issue the command:

```
GRANT SELECT ON EMPLOYEE, DEPARTMENT
        TO A3 WITH GRANT OPTION;
```

- A3 can grant the **SELECT** privilege on the **EMPLOYEE** relation to A4 by issuing:

```
GRANT SELECT ON EMPLOYEE TO A4;
```

  - Notice that A4 can't propagate the SELECT privilege because GRANT OPTION was not given to A4.

An Example (5)

- Suppose that A1 decides to revoke the SELECT privilege on the EMPLOYEE relation from A3; A1 can issue:

  ```
  REVOKE SELECT ON EMPLOYEE FROM A3;
  ```

- The DBMS must now **automatically revoke the SELECT privilege on EMPLOYEE from A4**, too, because A3 granted that privilege to A4 and A3 does not have the privilege any more.

An Example (6)

- Suppose that A1 wants to give back to A3 a limited capability to SELECT from the EMPLOYEE relation and wants to allow A3 to be able to propagate the privilege.
  - The limitation is to retrieve only the NAME, BDATE, and ADDRESS attributes and only for the tuples with DNO=5.
- A1 then create the view:

```
CREATE VIEW A3_EMPLOYEE AS
SELECT NAME, BDATE, ADDRESS
FROM EMPLOYEE WHERE DNO = 5;
```

- After the view is created, A1 can grant **SELECT** on the view *A3_EMPLOYEE* to A3 as follows:

```
GRANT SELECT ON A3_EMPLOYEE
TO A3
WITH GRANT OPTION;
```

An Example (7)

- Finally, suppose that A1 wants to allow A4 to update only the SALARY attribute of EMPLOYEE;
- A1 can issue:

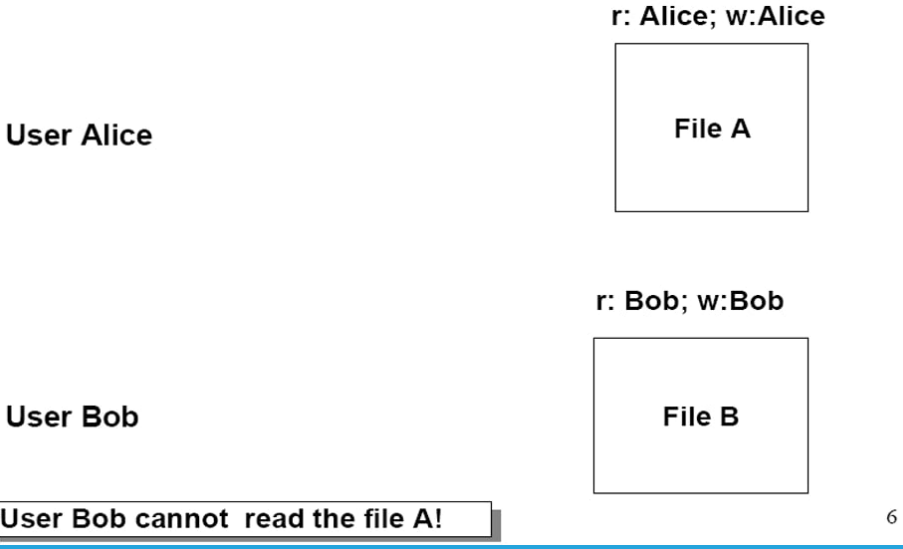
```
GRANT UPDATE ON EMPLOYEE (SALARY) TO A4;
```

- The **UPDATE** or **INSERT** privilege can specify particular attributes that may be updated or inserted in a relation.
- Other privileges (**SELECT**, **DELETE**) are not attribute specific.

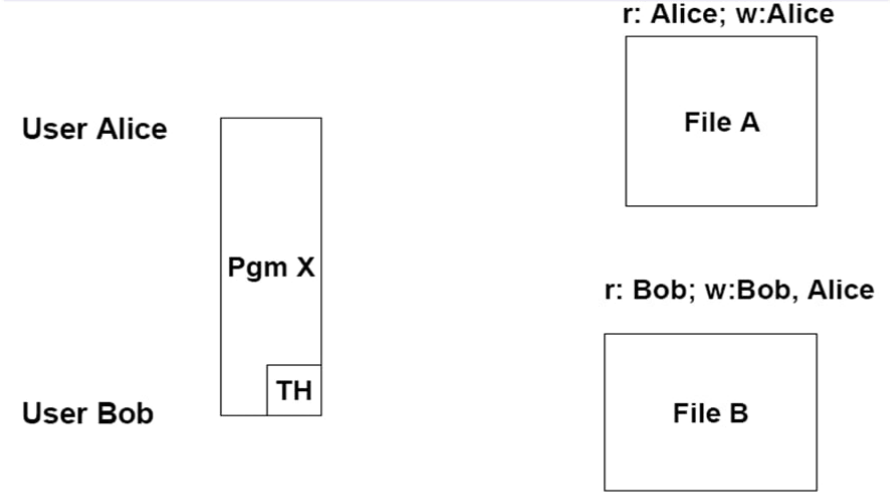
Inherent weakness of DAC

- **Unrestricted DAC** allows information from an object which can be read by a subject to be written to any other object.
  - Bob is denied access to file Y, so he asks Alice to copy Y to X that he can access.
- Suppose our users are trusted not to do this deliberately.
- It is still possible for **Trojan Horses** to copy information from one object to another.

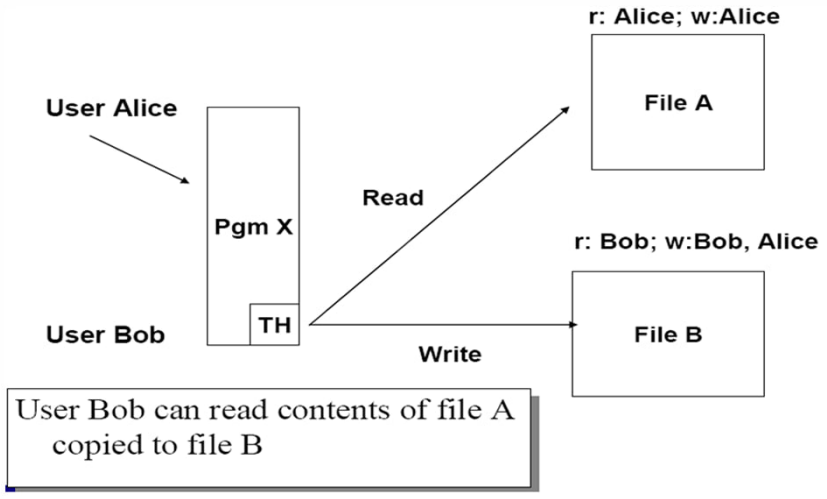
Trojan horse Example (1)



Trojan horse Example (2)



Trojan horse Example (3)



Mandatory Access Control (MAC)

Contents

1. Introduction to Database Security Issues
2. Discretionary Access Control (DAC)
3. **Mandatory Access Control (MAC)**
4. Role-Based Access Control (RBAC)
5. Attribute-Based Access Control (ABAC)
6. Context-Based Access Control (CBAC)

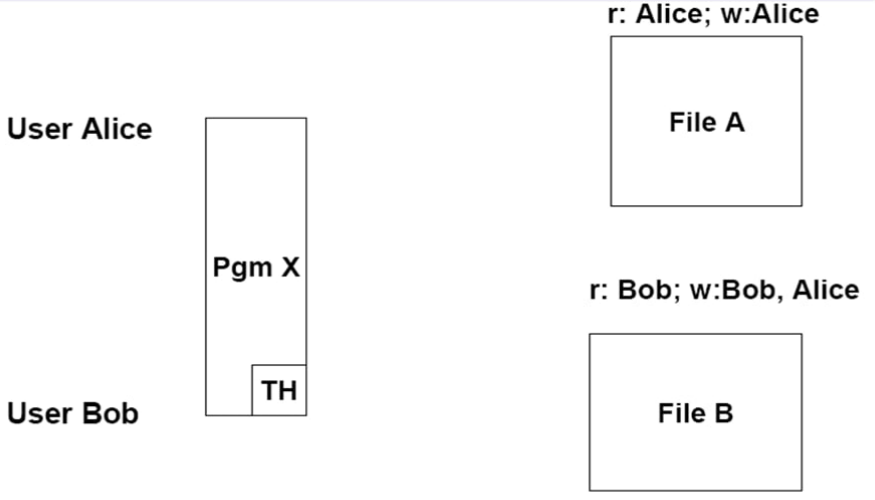
- Granting access to the data on the basis of **users' clearance level** and the **sensitivity level of the data**.
- **Bell-LaPadula's two principles**:
  - No read-up
  - No write-down secrecy.

- Typical security classes are top secret (TS), secret (S), confidential (C), and unclassified (U), where TS is the highest level and U is the lowest one:

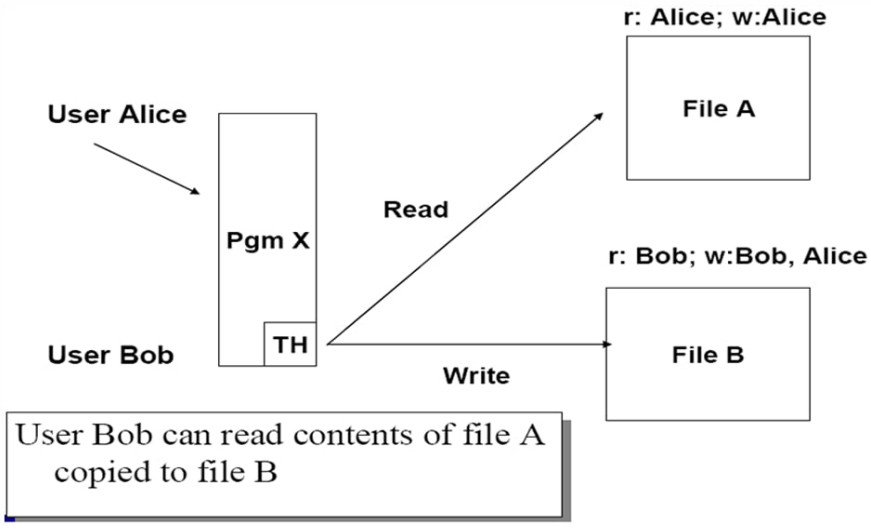
$$TS \geq S \geq C \geq U$$

- Two restrictions are enforced on data access based on the subject/object classifications:
  1. A subject S is not allowed **read access** to an object O unless **class(S)** $\geq$ **class(O)**.
     $\rightarrow$ This is known as the **simple security property**.
  2. A subject S is not allowed to **write** an object O unless **class(S)** $\leq$ **class(O)**.
     $\rightarrow$ This known as the **star property** (or *** property**).

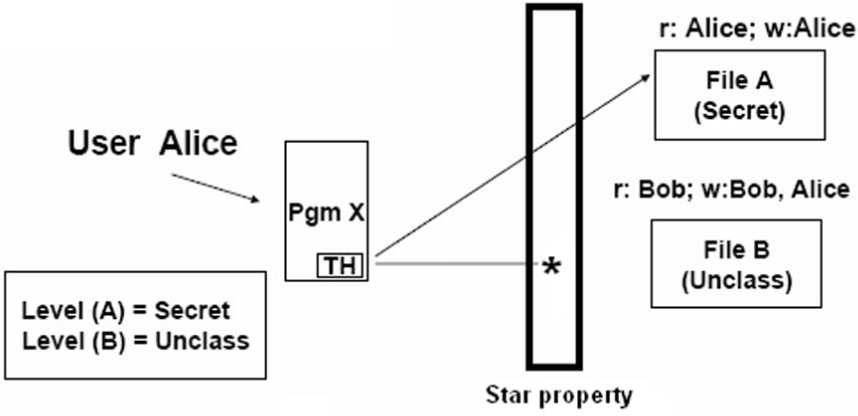
Why star property?



Why star property?



Why star property?



Multilevel relation (1)

- **Multilevel relation**: MAC + relational database model.
- **Data objects**: attributes and tuples.
- Each attribute A is associated with a **classification attribute C**.
- A **tuple classification attribute TC** is to provide a classification for each tuple as a whole, the highest of all attribute classification values.

$$R(A_1, C_1, A_2, C_2, ..., A_n, C_n, TC)$$

- The **apparent key** of a multilevel relation is the set of attributes that would have formed the primary key in a regular (single-level) relation.

EMPLOYEE

Name	Salary	Job Performance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Brown **C**	80000 **S**	Good **C**	**S**

(U: Unclassified, C: Confidential, S: Secret)

- A multilevel relation will appear to contain **different data** to subjects (users) with **different security levels**.

```
SELECT * FROM EMPLOYEE
```

EMPLOYEE:

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Brown **C**	80000 **S**	Good **C**	**S**

- View for a user with security level **S** (Secret):

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Brown **C**	80000 **S**	Good **C**	**S**

```
SELECT * FROM EMPLOYEE
```

EMPLOYEE table:

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Brown **C**	80000 **S**	Good **C**	**S**

- View for a user with security level **C** (Confidential):

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	NULL **C**	**C**
Brown **C**	NULL **C**	Good **C**	**C**

```
SELECT * FROM EMPLOYEE
```

EMPLOYEE table:

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Brown **C**	80000 **S**	Good **C**	**S**

- View for a user with security level **U** (Unclassified):

Name	Salary	JobPerformance	TC
Smith **U**	NULL **U**	NULL **U**	**U**

Properties of Multilevel relation (1)

- **Read and write operations**: satisfy the:
  - **No Read-Up** principle.
  - **No Write-Down** principle.

Properties of Multilevel relation (2)

- **Entity integrity**: all attributes that are members of the **apparent key** must not be null and must have the **same security classification** within each individual tuple.
- In addition, all other attribute values in the tuple must have a security classification **greater than or equal to** that of the apparent key.
- This constraint ensures that a user can see the key if the user is permitted to see any part of the tuple at all.

Properties of Multilevel relation (3)

- **Polyinstantiation**: where several tuples can have the **same apparent key value** but have **different attribute values** for users at different classification levels.

Polyinstantiation example (1)

EMPLOYEE

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Brown **C**	80000 **S**	Good **C**	**S**

- A user with security level **C**:

EMPLOYEE

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	NULL **C**	**C**
Brown **C**	NULL **C**	Good **C**	**C**

(Values with higher security level are shown as NULL)

Polyinstantiation example (2)

- A user with security level **C** executes:

```
UPDATE EMPLOYEE
SET Job_performance = 'Excellent'
WHERE Name = 'Smith';
```

Resulting Table (Polyinstantiation occurs):

Name	Salary	JobPerformance	TC
Smith **U**	40000 **C**	Fair **S**	**S**
Smith **U**	40000 **C**	Excellent **C**	**C**
Brown **C**	80000 **S**	Good **C**	**S**

Polyinstantiation of the Smith tuple.

Polyinstantiation example (3)

- A user with security level **U** (Unclassified) executes:

```
INSERT INTO EMPLOYEE
VALUES ('Brown', 50000, 'Average');
```

Resulting Table (Polyinstantiation occurs):

Name	Salary	JobPerformance	TC
Brown **U**	50000 **U**	Average **U**	**U**
Brown **C**	80000 **S**	Good **C**	**S**

- If the DBMS rejects the INSERT, the **U** user would deduce that 'Brown' exists at a higher level (Information Leak!).
- Allowing it closes this **covert channel**.

Polyinstantiation example (4)

- Continuing from the previous state, a user with security level **U** executes:

```
DELETE FROM EMPLOYEE
WHERE Name = 'Brown';
```

Resulting Table:

Name	Salary	JobPerformance	TC
Brown **C**	80000 **S**	Good **C**	**S**

(The U-level 'Brown' tuple is deleted, but the higher-level tuple survives)

- The DBMS **only deletes the tuple visible at level U**.
- **Conclusion:** Lower-level subjects cannot destroy or modify objects they do not have

- **Discretionary Access Control (DAC)** policies are characterized by a **high degree of flexibility**, which makes them suitable for a large variety of application domains.
  - The **main drawback** of DAC models is their **vulnerability to malicious attacks**, such as **Trojan horses** embedded in application programs.

Comparing DAC and MAC (2)

- By contrast, **mandatory policies** ensure a **high degree of protection** in a way, they prevent any illegal flow of information.
- Mandatory policies have the drawback of being **too rigid** and they are only applicable in **limited environments**.
- In many practical situations, **discretionary policies are preferred** because they offer a better **trade-off** between security and applicability.

Role-Based Access Control (RBAC)

Contents

1. Introduction to Database Security Issues
2. Discretionary Access Control (DAC)
3. Mandatory Access Control (MAC)
4. **Role-Based Access Control (RBAC)**
5. **Attribute-Based Access Control (ABAC)**
6. Context-Based Access Control (CBAC)

Role-Based Access Control (1)

- Role-based access control (RBAC) emerged rapidly in the 1990s as a proven technology for managing and enforcing security in large-scale enterprise-wide systems.
- Its basic notion is that **permissions are associated with roles**, and **users are assigned to appropriate roles**.
- Roles can be created using the **CREATE ROLE** and **DESTROY ROLE** commands.
  - The **GRANT** and **REVOKE** commands discussed under DAC can then be used to assign and revoke privileges from roles.

Role-Based Access Control (2)

- RBAC appears to be a **viable alternative** to traditional discretionary and mandatory access controls; it ensures that only authorized users are given access to certain data or resources.
- Many DBMSs have allowed the **concept of roles**, where privileges can be assigned to roles.
- **Role hierarchy** in RBAC is a natural way of organizing roles to reflect the organization's lines of authority and responsibility.

Role-Based Access Control (3)

- Another important consideration in RBAC systems is the possible **temporal constraints** that may exist on roles, such as time and duration of role activations, and timed triggering of a role by an activation of another role.
- Using an RBAC model is highly desirable goal for addressing the key security requirements of **Web-based applications**.
- In contrast, DAC and MAC models lack capabilities needed to support the security requirements emerging enterprises and Web-based applications.

Attribute-Based Access Control (ABAC)

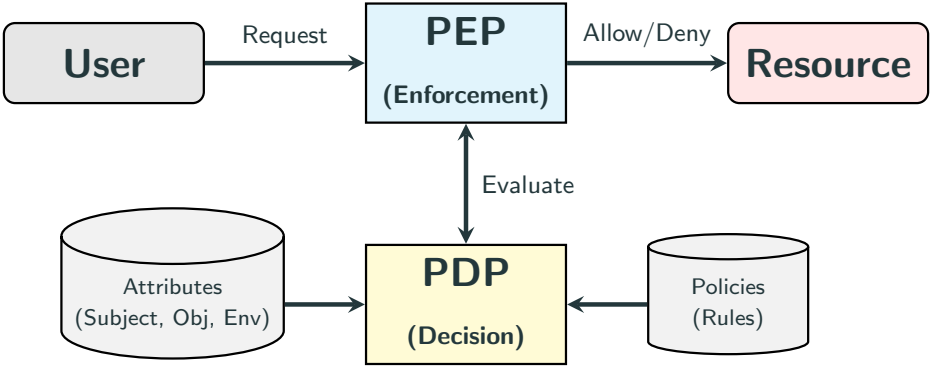
Attribute-Based Access Control (ABAC) (1)

- **Attribute-Based Access Control (ABAC)** is a logical access control model that evaluates **attributes** (characteristics or properties) rather than roles.
- Access rights are granted to users through the use of **policies** which evaluate:
  - **Subject attributes**: details about the user (e.g., job title, department, clearance level).
  - **Object attributes**: details about the data/resource (e.g., document type, classification, creation date).
  - **Environment conditions**: context of the access request (e.g., time of day, IP address, location).

Attribute-Based Access Control (ABAC) (2)

- **RBAC** uses predefined **roles**. It can become difficult to manage when the number of roles explodes ("Role Explosion").
- **ABAC** is highly **dynamic and flexible**, allowing for **fine-grained** access control without creating countless specific roles.
  - **Example Policy**:
    "Only managers **(Subject)** *can read* **(Action)** *financial reports* **(Object)** *during business hours* **(Environment)**.*"*

ABAC Architecture (How it works)



Context-Based Access Control (CBAC)

Core Concept

- **CBAC** is an access control model that regulates access based on the **environmental context** or state of the system at the time of the request.
- While RBAC asks "Who are you?"and ABAC asks "What are your attributes?", CBAC emphasizes the **"Situation"** (Context).
- **Principle:** A user might have valid permissions, but if the request occurs in an abnormal context (e.g., wrong time, strange location), access is denied.

Key Contextual Factors

- **Time:** Allowing access only during business hours or specific working shifts.
- **Location:** Checking internal IP addresses, GPS coordinates, or geographical boundaries (Geofencing).
- **Device:** Distinguishing company-issued devices (Managed) from personal devices (BYOD), or checking security patch levels.
- **Network:** Requiring a secure VPN connection or access from a wired internal network.

Comparing Access Control Models

Model	Evaluation Focus	Characteristics
RBAC	Roles	Static, easy to manage but lacks flexibility.
ABAC	Attributes	Highly granular but can be complex to implement.
CBAC	**Environment (Context)**	**Dynamic, enhances real-time security.**

Practice: CBAC is often used as an additional security layer on top of RBAC or ABAC.

Benefits and Practical Scenario

- **Benefit:** Mitigates Account Takeover attacks. Even with a stolen password, hackers are blocked if accessing from an unusual country or unrecognized device.
- **Banking Scenario:**
  - A loan officer can only view customer records when at the **branch** and during **business hours**.
  - If accessed from **home** at **2 AM**, the system automatically locks the access.
- **Trend:** Serves as a foundation for the modern **Zero Trust**

Thank You!

for your listening and attention.

Q & A

Do you have any questions for Group 6?